DataPulse — Team 9

TEAM CONFIGURATION

Role	Name	Email	Location
Backend with Python	Joseph Lubandi	joseph.lubandi@amalitech.com	Rwanda
Backend with Python	Diane Ishimwe	diane.ishimwe@amalitech.com	Rwanda
Backend with Python	Kirenga Joe Bright	bright.kirenga@amalitech.com	Rwanda
Quality Assurance	Bernice Adime Mawuena	bernice.mawuena@amalitech.com	Takoradi
DevOps	Yram Asher Tetteh-Abotsi	yram.tetteh-abotsi@amalitech.com	Takoradi
Data Engineering	Zainab Abdullai	zainab.abdullai@amalitech.com	Kumasi

Product Owner: Kevin

Scrum Master: Selected by PO from team members

PROBLEM STATEMENT

Organizations struggle to maintain data quality across their datasets. DataPulse is a data quality monitoring tool that allows users to upload CSV/JSON files, define validation rules, run quality checks that produce a quality score (0-100), generate quality reports, and track quality trends over time through a dashboard.

MVP REQUIREMENTS

- Upload CSV and JSON data files for validation
- Define validation rules: null checks, data type checks, range checks, uniqueness checks
- Run quality checks producing a quality score from 0 to 100 (percentage of rows passing all rules)
- Generate quality reports with detailed findings per rule
- Trend dashboard showing quality scores over time per dataset
- REST API for all operations

Stretch Goals

- Scheduled quality checks (cron-based)
- Email alerts when quality drops below threshold

TECH STACK

- Backend: Python 3.11+ / FastAPI / SQLAlchemy / PostgreSQL / Pydantic
- Data Processing: Pandas for data validation and analysis
- Testing: pytest with comprehensive test coverage
- Infrastructure: Docker / Docker Compose / GitHub Actions CI/CD

DEFAULT CREDENTIALS

Role	Email	Password
ADMIN	admin@amalitech.com	password123
USER	user@amalitech.com	password123

STARTER CODE

Location: https://github.com/AmaliTech-Training-Academy/Phase1-Group-Projects/tree/126e143b8dd03752be797fede9f538436cbf3b91/3-DataPulse

- Review the README.md for setup instructions
- Look for TODO comments in the codebase for implementation guidance
- Run with: docker-compose up --build

ROLE-SPECIFIC DELIVERABLES

Backend Developer 1 (File Upload & Validation Engine)

Deliverable	Details
File Upload API	Endpoints for CSV and JSON file upload with validation
Validation Rules Engine	4 rule types: null check, type check, range check, uniqueness
Quality Score Calculator	Compute score as % of rows passing all defined rules
Rule Management API	CRUD for validation rules per dataset
File Parsing	Robust CSV/JSON parsing with error handling

Backend Developer 2 (API & Reporting)

Deliverable	Details
Quality Report API	Detailed report generation with per-rule findings
Trend API	Historical quality scores per dataset over time
Dashboard API	Aggregate endpoints for dashboard (latest scores, trends)
Auth System	JWT-based authentication and authorization
Database Schema	PostgreSQL schema for datasets, rules, results, reports

Backend Developer 3 (Scheduling & Notifications)

Deliverable	Details
Scheduled Checks	Cron-based automatic quality check execution
Notification System	Alert when quality score drops below threshold
Batch Processing	Run checks on multiple datasets efficiently
Audit Log	Track all quality check runs and results
Configuration API	Manage schedules and alert thresholds

Data Engineer 1 (Schema & Pipeline)

Deliverable	Details
Database Design	Optimized schema for quality data storage
Data Pipeline	ETL pipeline for quality metrics aggregation
Index Optimization	Database indexes for query performance
Migration Scripts	Database migration and seed scripts
Data Documentation	Complete data dictionary and schema docs

QA Engineer

Deliverable	Details
Test Plan	Comprehensive test plan covering all MVP features

API Tests	pytest tests for all endpoints
Validation Tests	Test each rule type with edge cases
Integration Tests	End-to-end upload, validate, report flow
Bug Reports	Documented bugs with severity and steps to reproduce

DevOps Engineer

Deliverable	Details
Docker Setup	Dockerfile for FastAPI application and database
Docker Compose	Multi-container setup with proper networking
CI/CD Pipeline	GitHub Actions for build, test, deploy
Environment Config	Dev and test environment configurations
Deployment Docs	Setup and deployment documentation

DAILY MILESTONES

Day 1: Setup & Foundation

- Clone repository, review starter code and README
- Set up development environment (Docker, IDE)
- Database schema finalized and migrated
- Auth system (registration/login) working
- CI/CD pipeline initial setup

Day 2: Core Features

- Primary CRUD operations complete and tested
- Core business logic implemented
- Frontend/UI connected to backend APIs
- Initial test suite running
- Sample data loaded

Day 3: Integration & Testing

- All MVP features implemented
- End-to-end integration tested
- Analytics/dashboard data pipeline working
- Bug fixes from testing

- Docker containers all running together

Day 4: Polish & Preparation

- All critical bugs fixed
- Final testing and quality assurance
- Documentation complete
- Presentation preparation and demo rehearsal
- Stretch goals if time permits

COLLABORATION GUIDELINES

Daily Standup

Brief daily standup meeting: what you did yesterday, what you plan today, any blockers.

Project Management

- Use GitHub Projects for task tracking (Kanban board)
- Track all tasks with clear assignee and status

Version Control

- Use Git with feature branches (feature/feature-name)
- Create pull requests for code review before merging
- Write meaningful commit messages
- Never push directly to main branch

Shared Project Log

Maintain a shared document logging daily progress, decisions made, and blockers encountered.